Assignment for SDE I

Submission Guidelines:

Format: Share a public GitHub repository containing:

- **/backend** and **/frontend** folders (or monorepo structure)
- README.md with setup steps, environment variable info, and any assumptions you made
- Include sample .env.example
- A deployed live link

Access: Ensure the repository is public (a private repository will lead to disqualification)

Timeline: Expected time is ideally 4–6 hours (2–3 hours more if you attempt the stretch section). Please submit by the coming Monday and can submit early if completed.

Meme'd Campaign Payout Tracker

Overview

Meme'd is a creator marketplace. Brands run campaigns, and creators (meme pages, clip accounts, hype pages) post content for those campaigns on Instagram or YouTube. Creators get paid through a milestone ladder. For example: 10K views → ₹500, 50K views → ₹2,000, 100K views → ₹5,000. A creator's payout is the amount of the highest milestone their post has crossed, and total payouts must always stay within the brand's campaign budget.

Build a web app where brands can create campaigns and creators can submit posts and track their earnings.

Requirements

1. **Backend (Node.js + TypeScript + Express + MongoDB with Mongoose)**
   a. Auth
      - JWT-based signup and login
      - Two roles: brand and creator
   b. Brand
      - Create a campaign: name, platform (Instagram / YouTube), total budget, milestone ladder (view threshold + payout pairs)
      - View each campaign's budget spent vs. remaining
   c. Creator
      - List live campaigns
      - Submit a post link to a campaign
      - View their total earnings
   d. Views and payouts
      - PATCH /api/submissions/:id/views: update a post's view count (this simulates platform data; no role check needed on this endpoint)
      - Calculate the creator's payout based on the highest milestone crossed
      - A campaign must never pay out more than its remaining budget
2. **Frontend (Next.js + TypeScript)**

Assignment for SDE I

a. Login / Signup
b. Brand page: create a campaign, see budget spent vs. remaining
c. Creator page: browse campaigns, submit a post, see earnings
d. Styling can be minimal. Focus on functionality and structure.

3. **Testing**
   a. Use Jest
   b. Write unit tests for payout calculation and the budget cap
4. **Deployment**
   a. Free tiers (e.g. MongoDB Atlas, Vercel, Render) are completely fine
5. **Bonus (Optional)**
   a. Cache the campaign list with Redis (doesn't need to be deployed)
   b. Flag a submission when its views spike unusually fast
6. **Stretch (Optional)**
   a. Ensure two simultaneous view updates can never overspend the campaign budget, and repeated updates never double-pay
   b. Dockerise the app
   c. Add a short note (half a page max): how would this scale to 10,000 creators?

Where the brief is open (e.g. a post crosses two milestones in a single update, or the budget runs out mid-payout), make a call and note it in your README.

What We're Evaluating

- Working end-to-end build (deployed)
- Correct payout and budget logic
- Code structure, readability and TypeScript usage
- Proper use of async/await and error handling
- Quality of git commits
- Clear README and stated assumptions

Important Notes

- **Got questions?** Reach out at hiring@wldd.in before the deadline if you need clarification.
- **We're evaluating holistically:** Attention to detail, real-world thinking, clarity of communication, and how you handle messy situations matter more than perfect answers.
- **What's next:** In the follow-up call, you'll walk us through your code and make a small change live, so make sure you understand every part of what you submit.